Fase 2 — División del trabajo entre los 4 desarrolladores

Guía de trabajo para construir el **Punto 2 (pesaje inicial / tara)** entre cuatro personas, en la misma rama. Dice dónde está parado cada uno respecto de la Fase 1, qué tiene que hacer, cómo hacerlo, cómo probarlo y trae el prompt para usar con Claude Code.

- **Fecha:** 22/10/2026 · **Rama:** `fase-2-pesaje-inicial` (sale de `main`, que ya incluye la Fase 1)
- **Fase autorizada por el equipo** el 22/10/2026. Hasta que se cierre la fase, la línea de `CLAUDE.md` que dice "Fase 2: NO iniciada" queda desactualizada (la corrige la Etapa D al final).
- **Primera entrega del TP:** 10/11/2026.

Etapa	Responsable	Usuario de GitHub	Qué construye
A	Martín González	`mgonzalez309-dev`	Modelo y piezas base
B	Tomás Calvo	`tomas-calvo01`	Regla de negocio
C	Benjamín Alasino	`benjaalasino`	Endpoint de la balanza (TMS)
D	Joaquín Contreras	`joacoContreras`	Pruebas, Postman y documentación

---

1. Dónde estamos parados (lo que dejó la Fase 1)

La Fase 1 construyó el Punto 1: SAP manda una orden y el sistema la crea **en estado 1**. Esto ya existe y se reutiliza:

Ya existe	Dónde	Cómo se usa en la Fase 2
La orden y su estado	`model/Order.java`, `model/OrderStatus.java`	Se le agregan 3 campos. El estado 2 ya está en el enum (`INITIAL_WEIGHING_REGISTERED`) pero **nada lo usa todavía**
Cómo guardar el estado como 1 a 4	`model/OrderStatusConverter.java`	No se toca
El historial de estados	`model/OrderStatusLog.java` y `OrderBusiness.logStatus(...)` (método privado)	Se llama al cambiar de 1 a 2
Buscar una orden por número	`OrderBusiness.load(orderNumber)` → lanza `NotFoundException`	Primer paso de la regla
Excepciones de negocio	`business/exception/` (`NotFound`, `Found`, `BadRequest`, `Business`)	Se suma una nueva para "estado incorrecto"
Un controller de integración como modelo	`integration/sap/controllers/OrderSapRestController.java`	Se copia el estilo para el controller de la balanza
El usuario del token	`BaseRestController.getUserLogged()`	Es el "actor" del historial
Las URLs	`controllers/Constants.java`	Se agrega la de TMS
Swagger	`config/OpenApiConfig.java` y anotaciones `@Operation`	El endpoint nuevo se documenta igual
Postman	`docs/postman/IW3-Gas-Terminal.postman_collection.json` (10 pedidos, 19 verificaciones)	Se agrega una carpeta nueva; **lo de la Fase 1 no debe romperse**

Lo que NO existe todavía y se construye ahora: los campos de tara en la orden, la contraseña de activación, la excepción de "estado incorrecto", la regla de la tara, el endpoint de la balanza y las pruebas de todo eso.

2. Qué hay que construir (la Fase 2 en una página)

La balanza (TMS) manda el peso del camión vacío. El backend:

1. Busca la orden → si no existe, **404**.
2. Comprueba que la tara sea mayor que 0 → si no, **400**.
3. Comprueba que la orden esté en estado 1 → si no, **409**.
4. Guarda la tara y la fecha y hora del pesaje.
5. Genera una **contraseña de 5 dígitos** (10000 a 99999) con `SecureRandom`.
6. Pasa la orden al **estado 2** y anota el cambio en el historial (de 1 a 2, con el usuario del token).
7. Responde la contraseña.

Todo en **una transacción**: si algo falla, la orden queda como estaba.

```
Balanza (TMS) ──POST /api/v1/integration/tms/orders/{n}/tare  { "weight": 14500 }──►
   OrderTmsRestController (C) ──► IOrderBusiness.registerTare (B) ──► Order + OrderStatusLog (A)
◄── 200 { "activationPassword": 48213 }
```

Criterios de aceptación de la fase

#	Criterio
1	Tara válida: 200, contraseña de 5 dígitos y la orden pasa a estado 2 con tara y fecha guardadas
2	El historial tiene 2 registros: `null→1` y `1→2`, con el usuario como actor
3	Orden inexistente: 404
4	Tara 0, negativa o ausente: 400
5	Orden que no está en estado 1 (por ejemplo, una segunda tara): 409
6	Sin token: 401
7	`GET /orders/{n}` **no muestra** la contraseña
8	Atomicidad: si falla a mitad de camino, la orden no cambia
9	El endpoint figura en Swagger con sus respuestas
10	Postman: la carpeta de la Fase 2 pasa con newman **y** la de la Fase 1 sigue pasando (19 verificaciones)
11	Informe y guía de la fase, y PDF

3. El contrato compartido (no se cambia sin avisar a los 4)

Estas decisiones son las que permiten que las cuatro partes encajen. Se tomaron con la recomendación del análisis de la fase; si el equipo quiere cambiar alguna, **hay que hacerlo antes de repartir los prompts**.

#	Tema	Resolución
1	Código HTTP de "estado incorrecto"	**409 Conflict** (`ConflictException`)
2	Cuerpo del pedido	`{ "weight": 14500.0 }` (kg)
3	Respuesta exitosa	**200** con `{ "activationPassword": 48213 }`
4	Firma del método de negocio	`Order registerTare(Integer orderNumber, double weight, String actor) throws NotFoundException, BadRequestException, ConflictException, BusinessException`
5	La contraseña en `GET /orders`	**No se publica** (`@JsonIgnore`): es un secreto entre la balanza y el sistema de carga

#	Tema	Resolución
6	Orden de las comprobaciones	404 (orden) → 400 (tara) → 409 (estado)
7	Campos nuevos en `Order`	`tare` (`Double`), `initialWeighingDateTime` (`LocalDateTime`), `activationPassword` (`Integer`). **Todos nulables**: la tabla `orders` ya tiene filas y `ddl-auto=update` solo agrega columnas
8	URL del endpoint	`POST /api/v1/integration/tms/orders/{orderNumber}/tare`
9	Observación del historial	`"Tara registrada: <peso> kg"`

4. Quién toca qué

Regla de oro: **cada archivo tiene un solo dueño**. Si necesitás un cambio en un archivo ajeno, se lo pedís a su dueño; no lo editás vos.

Archivo	Acción	Dueño
`model/Order.java`	Modificar (3 campos)	**A**
`business/exception/ConflictException.java`	Crear	**A**
`controllers/Constants.java`	Modificar (1 línea)	**A**
`business/ActivationPasswordGenerator.java`	Crear	**A**
`src/test/.../business/ActivationPasswordGeneratorTest.java`	Crear	**A**
`docs/guia-estudio-src.md`, su PDF y `docs/pdf/README.md`	Commitear lo pendiente al abrir la rama	**A** (después los actualiza D)
`business/IOrderBusiness.java`	Modificar (1 método)	**B**
`business/OrderBusiness.java`	Modificar (1 método)	**B**
`integration/tms/controllers/OrderTmsRestController.java`	Crear	**C**
`integration/tms/model/TareRequest.java` y `TareResponse.java`	Crear	**C**
`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Agregar carpeta	**D**
`docs/fase-2-pesaje-inicial.md` y su PDF	Crear	**D**
`CLAUDE.md`, `docs/plan-de-trabajo.md`, `docs/guia-estudio-src.md` (y sus PDF)	Actualizar al cierre	**D**

No se commitea la carpeta `.postman/` (es el vínculo de la colección con el espacio de trabajo de Postman de cada uno).

5. Orden de trabajo y de pushes

Las partes se apoyan unas en otras: C no compila sin la interfaz de B, y B no compila sin los campos de A. Por eso hay un orden mínimo de pushes; el resto del tiempo se trabaja en paralelo.

```
Etapa 0  A crea la rama y publica la documentación (≈15 min)    ← los demás esperan este push
         A implementa y pushea sus piezas (≈1 h)
Etapa 1  B pushea la firma de registerTare + una implementación mínima que compile (≈30 min)
         ── desde acá C y D compilan ──
Etapa 2  B completa la regla · C termina el endpoint · D arma Postman  (en paralelo)
Etapa 3  Integración: prueba de punta a punta con los 4 presentes + documentación
Etapa 4  Revisión cruzada y PR a main con 3 aprobaciones
```

Mientras esperan un push, los demás **no pierden el tiempo**: B lee `OrderBusiness`, C estudia `OrderSapRestController`, D diseña los casos de Postman. Los tiempos son estimaciones.

Revisión cruzada (para la evaluación individual)

La nota es individual y cada integrante tiene que poder explicar el circuito completo. Antes del PR, cada uno revisa la parte de otro: **A revisa a B, B revisa a C, C revisa a D, D revisa a A**.

6. Cómo trabajar con Git

```
# Solo Martín (A), una vez:
git checkout main && git pull
git checkout -b fase-2-pesaje-inicial
# ... commit de la documentación ...
git push -u origin fase-2-pesaje-inicial

# Los otros tres, cuando Martín avise que la rama está publicada:
git fetch
git checkout fase-2-pesaje-inicial

# Todos, antes de CADA push:
git pull --rebase
git push
```

Regla	Detalle
Commits	Atómicos, de **100 líneas o menos**, en español, con formato `tipo(ámbito): descripción` (`feat`, `fix`, `docs`, `test`, `build`, `refactor`)
Coautoría	**Sin** `Co-Authored-By: Claude` ni "Generated with Claude Code" (regla del equipo en `CLAUDE.md`)
`main`	Nadie pushea a `main`. Solo entra por Pull Request
Force push	Prohibido
Rama de la fase	Pushear acá es libre y no pide PR (la protección rige solo para `main`)

Si el push es rechazado ("non-fast-forward" / "fetch first"): no es un error. Alguien pusheó antes. Hacé `git pull --rebase` y volvé a pushear.

Si hay un conflicto: significa que dos personas tocaron las mismas líneas, o sea que alguien editó un archivo ajeno. No lo resuelvas a ciegas: avisen y decidan entre los dos dueños.

Excepción de los 100 líneas: el JSON de Postman es un archivo generado y puede superar ese límite. Va en un commit aparte, solo con ese archivo.

7. Cómo probar en tu máquina

Cada uno trabaja con su propia base local.

```
docker compose up -d                 # la base (contenedor iw3-gas-mysql)
./mvnw spring-boot:run               # la app en http://localhost:8080
# PowerShell, solo la primera vez, con la app ya iniciada:
Get-Content docs/sql/seed-dev-users.sql | docker exec -i iw3-gas-mysql mysql -uroot -proot iw3_db
```

Usuario de desarrollo: `admin` / `admin123`. Swagger: `http://localhost:8080/swagger-ui.html`.

Hibernate crea las columnas nuevas solo, al arrancar (`ddl-auto=update`).

Etapa A — Modelo y piezas base

Responsable: Martín González · **Estimación:** ≈1 h 15 min · **Es la primera: los demás dependen de su push.**

Punto de partida (qué heredás de la Fase 1)

`Order` ya existe con su estado, pero **no tiene** campos para la tara. Hay excepciones para 404, 302, 400 y 500 pero **ninguna para "estado incorrecto"**. `Constants` tiene la URL de SAP y la de órdenes, **no la de la balanza**. Tu copia de `main` tiene sin commitear la guía de estudio de `src`.

Objetivo

Dejar listas las piezas que usan las otras tres partes: los campos nuevos de la orden, la excepción de 409, la URL de TMS y el generador de contraseñas.

Qué hacer, paso a paso

Paso 0 — Abrir la rama y publicar la documentación (primero de todo).

1. Desde `main` actualizado: `git checkout -b fase-2-pesaje-inicial`.
2. Commitear, **en commits separados y sin incluir `.postman/`**:
   - `docs/guia-estudio-src.md`, `docs/pdf/guia-estudio-src.pdf` y `docs/pdf/README.md`.
   - `docs/fase-2-division-de-trabajo.md` y su PDF.
3. Pushear la rama y **avisar a los otros tres**.

Paso 1 — Campos nuevos en `model/Order.java`.

```
/** Peso del camión vacío (kg), enviado por la balanza. Nulo hasta el pesaje inicial. */
@Schema(accessMode = Schema.AccessMode.READ_ONLY)
@Column
private Double tare;

/** Momento del pesaje inicial. */
@Schema(accessMode = Schema.AccessMode.READ_ONLY)
@Column
private LocalDateTime initialWeighingDateTime;

/** Contraseña de activación de 5 dígitos. Secreto entre la balanza y el sistema de carga: no se publica. */
@JsonIgnore
@Column
private Integer activationPassword;
```

Tienen que ser **nulables** (`Double` e `Integer`, no `double` ni `int`).

Paso 2 — `business/exception/ConflictException.java`. Copiá el molde de `BadRequestException` (builder incluido) y cambiá el nombre y el comentario ("estado incorrecto para la operación → 409").

Paso 3 — `controllers/Constants.java`. Una línea:

```
public static final String URL_INTEGRATION_TMS = URL_INTEGRATION + "/tms";
```

Paso 4 — `business/ActivationPasswordGenerator.java` (clase de paquete, como `OrderValidator`):

```
final class ActivationPasswordGenerator {
    private static final SecureRandom RANDOM = new SecureRandom();
    private ActivationPasswordGenerator() { }
    /** Entero de 5 dígitos: de 10000 a 99999. */
    static int generate() { return 10000 + RANDOM.nextInt(90000); }
}
```

Paso 5 — Test unitario en `src/test/java/ar/edu/iua/iw3/business/ActivationPasswordGeneratorTest.java`: generar muchas contraseñas (por ejemplo 10.000), comprobar que todas estén entre 10000 y 99999 y que no sean

siempre la misma. **Sin** `@SpringBootTest` (no debe necesitar la base).

Cómo probar

```
./mvnw -B -q -DskipTests compile
./mvnw -B -Dtest=ActivationPasswordGeneratorTest test
./mvnw spring-boot:run     # revisar en el log que Hibernate agrega las 3 columnas a "orders"
```

Con la app andando, `GET /api/v1/orders/{n}` tiene que **seguir funcionando** y no mostrar `activationPassword`.

Criterio de terminado

- Compila y el test pasa.
- La app arranca con una base que **ya tiene órdenes** y las columnas se agregan sin error.
- `GET /orders` no muestra la contraseña.
- Pusheado a la rama, y avisaste a B.

No tocar

`IOrderBusiness`, `OrderBusiness` (B), el paquete `integration` (C), Postman y `docs/` salvo lo del paso 0 (D).

Prompt para Claude Code (Etapa A)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 2 (pesaje inicial / tara), rama fase-2-pesaje-inicial. Somos 4 personas en la misma rama y cada archivo tiene un solo dueño. El equipo ya autorizó la fase: la línea de CLAUDE.md que dice "Fase 2 NO iniciada" quedó desactualizada; no la edites.

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md y docs/fase-2-division-de-trabajo.md (secciones 2, 3, 4 y "Etapa A").

Soy Martín González y hago la ETAPA A (modelo y piezas base).

Paso 0 (primero): desde main actualizado creá la rama fase-2-pesaje-inicial. Commiteá, en commits separados y SIN incluir la carpeta .postman/: (a) docs/guia-estudio-src.md, docs/pdf/guia-estudio-src.pdf y docs/pdf/README.md; (b) docs/fase-2-division-de-trabajo.md y su PDF. No pushees hasta que yo te lo pida.

Después implementá SOLO estos archivos:
1. src/main/java/ar/edu/iua/iw3/model/Order.java: agregar tare (Double), initialWeighingDateTime (LocalDateTime) y activationPassword (Integer). Los tres nulables (la tabla orders ya tiene filas). tare e initialWeighingDateTime con @Schema(accessMode = READ_ONLY); activationPassword con @JsonIgnore para que no salga en el JSON.
2. business/exception/ConflictException.java: mismo molde que BadRequestException (significa "estado incorrecto" y se traduce a 409).
3. controllers/Constants.java: agregar URL_INTEGRATION_TMS = URL_INTEGRATION + "/tms".
4. business/ActivationPasswordGenerator.java: clase final de paquete con un static int generate() que devuelve un entero de 5 dígitos (10000 a 99999) usando SecureRandom.
5. src/test/java/ar/edu/iua/iw3/business/ActivationPasswordGeneratorTest.java: JUnit 5 sin @SpringBootTest; comprobar rango 10000-99999 en muchas ejecuciones y que no sea siempre el mismo valor.

NO toques: IOrderBusiness, OrderBusiness, el paquete integration, Postman ni docs/ (salvo el paso 0). Si creés que hace falta cambiar algo ajeno, avisame en vez de hacerlo.

Reglas: seguí la metodología de la cátedra (capas, excepciones checked, Constants); identificadores en inglés y comentarios en español. Commits atómicos de 100 líneas o menos, en español, formato tipo(ámbito): descripción, SIN Co-Authored-By de Claude ni "Generated with Claude Code". No hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase. No amplíes el alcance ni avances a otra etapa. Antes de editar, mostrame tu plan de archivos y esperá mi OK.

Verificá de verdad: compilá (./mvnw -B -q -DskipTests compile), corré el test (./mvnw -B -Dtest=ActivationPasswordGeneratorTest test) y arrancá la app con la base levantada para ver que Hibernate agrega las 3 columnas y que GET /api/v1/orders/{n} no muestra activationPassword. Reportá solo lo que ejecutaste; si algo falla, decilo.
```

Etapa B — Regla de negocio

Responsable: Tomás Calvo · **Estimación:** ≈2 h · **Es la parte central de la fase.**

Punto de partida (qué heredás de la Fase 1)

`OrderBusiness` ya sabe recibir una orden (`add`), buscarla (`load`) y anotar el historial (`logStatus`, privado). **No hay ninguna operación que cambie el estado de una orden ya creada.** El enum ya tiene el estado 2. Esta es la **primera regla de transición de estado** del proyecto: lo que hagas acá es el molde de las Fases 3, 4 y 5.

Objetivo

Implementar `registerTare`: validar, guardar la tara, generar la contraseña, pasar la orden de 1 a 2 y registrar el cambio, todo en una transacción.

Qué hacer, paso a paso

Paso 1 — Etapa 1 (rápido, para no frenar a C y D). Cuando A haya publicado sus piezas (`git pull --rebase`):

- En `IOrderBusiness.java` agregá la firma:

```
/**
 * Registra la tara enviada por la balanza (punto 2): guarda el peso, genera la contraseña
 * de activación, pasa la orden del estado 1 al 2 y registra el cambio de estado.
 */
public Order registerTare(Integer orderNumber, double weight, String actor)
        throws NotFoundException, BadRequestException, ConflictException, BusinessException;
```

- En `OrderBusiness.java` una implementación **mínima que compile**:

```
@Override
public Order registerTare(Integer orderNumber, double weight, String actor)
        throws NotFoundException, BadRequestException, ConflictException, BusinessException {
    throw new UnsupportedOperationException("Pendiente: Etapa 2");
}
```

- Commit y push enseguida, y **avisar a C y D**.

Paso 2 — La regla real (Etapa 2). Reemplazá el cuerpo por algo con esta forma:

```
@Override
@Transactional(rollbackFor = Exception.class)
public Order registerTare(Integer orderNumber, double weight, String actor) throws ... {
    Order order = load(orderNumber);                                    // 404
    if (!(weight > 0)) {                                                // 400 (incluye NaN)
        throw BadRequestException.builder().message("weight debe ser mayor que 0 (kg)").build();
    }
    if (order.getStatus() != OrderStatus.PENDING_INITIAL_WEIGHING) {  // 409
        throw ConflictException.builder().message("La orden " + orderNumber
                + " no está pendiente de pesaje inicial (estado " + order.getStatus().getCode() + ")").build();
    }
    try {
        OrderStatus from = order.getStatus();
        order.setTare(weight);
        order.setInitialWeighingDateTime(LocalDateTime.now());
        order.setActivationPassword(ActivationPasswordGenerator.generate());
        order.setStatus(OrderStatus.INITIAL_WEIGHING_REGISTERED);
        order = orderDAO.save(order);
        logStatus(order, from, order.getStatus(), actor, "Tara registrada: " + weight + " kg");
        return order;
    } catch (Exception e) {
        log.error(e.getMessage(), e);
        throw BusinessException.builder().ex(e).build();
    }
}
```

Puntos delicados (lecciones de la Fase 1):

- Las comprobaciones 404, 400 y 409 van **fuera del** `try`. Si estuvieran adentro, el `catch (Exception)` las convertiría en `BusinessException` y saldría 500 en vez del código correcto.
- `@Transactional(rollbackFor = Exception.class)` es necesario porque las excepciones de negocio son *checked* y Spring no revierte por ellas.
- Usá el orden del contrato: 404 → 400 → 409.

Cómo probar

Podés probar sin el controller de C usando un test o la consola. Lo más simple es esperar a C y probar por HTTP:

```
TOKEN=$(curl -s -X POST localhost:8080/api/v1/login -d "username=admin&password=admin123")
curl -i -X POST localhost:8080/api/v1/integration/tms/orders/90001/tare \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{"weight":14500}'
```

(La orden 90001 hay que crearla antes, con *Crear orden* de Postman o Swagger.)

Casos que tenés que comprobar en la base y en `GET /orders/{n}/status-log`:

Caso	Esperado
Tara válida	200; la orden pasa a estado 2; el historial tiene `1→2`
Segunda tara sobre la misma orden	409; no cambia nada
`weight` 0 o negativo	400
Orden inexistente	404
Falla simulada al final	La orden queda en estado 1 y sin tara

Criterio de terminado

- Los 5 casos se comportan como dice la tabla.
- La contraseña guardada en la base tiene 5 dígitos.
- Pusheado a la rama y A revisó tu código.

No tocar

`Order.java`, `ConflictException`, `Constants` (A); el controller (C); Postman y docs (D).

Prompt para Claude Code (Etapa B)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 2 (pesaje inicial / tara), rama fase-2-pesaje-inicial. Somos 4 personas en la misma rama y cada archivo tiene un solo dueño. El equipo ya autorizó la fase: la línea de CLAUDE.md que dice "Fase 2 NO iniciada" quedó desactualizada; no la edites.

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md y docs/fase-2-division-de-trabajo.md (secciones 2, 3, 4 y "Etapa B"), y estudiá business/OrderBusiness.java (add, load, logStatus) y OrderValidator.

Soy Tomás Calvo y hago la ETAPA B (regla de negocio). Antes de empezar hacé git fetch, git checkout fase-2-pesaje-inicial y git pull --rebase: necesito que Martín (Etapa A) ya haya publicado los campos de Order (tare, initialWeighingDateTime, activationPassword), ConflictException y ActivationPasswordGenerator. Si no están, avisame y esperá.

Implementá SOLO estos dos archivos:
1. business/IOrderBusiness.java: agregar Order registerTare(Integer orderNumber, double weight, String actor) throws NotFoundException, BadRequestException, ConflictException, BusinessException.
2. business/OrderBusiness.java: implementar registerTare.

En DOS pasos: primero (Etapa 1) solo la firma y una implementación mínima que compile (lanza UnsupportedOperationException), la commiteamos y la pusheamos enseguida para no frenar a Benjamín (Etapa C) y a Joaquín (Etapa D). Después (Etapa 2) la regla real:
- @Transactional(rollbackFor = Exception.class).
- Orden de comprobaciones: load(orderNumber) (404 NotFoundException) → peso inválido con !(weight > 0) (400 BadRequestException) → estado distinto de PENDING_INITIAL_WEIGHING (409 ConflictException).
- Esas tres comprobaciones van FUERA del try/catch, para que el catch (Exception) no las convierta en BusinessException (500).
- Dentro del try: guardar tare, initialWeighingDateTime = ahora, activationPassword = ActivationPasswordGenerator.generate(), status = INITIAL_WEIGHING_REGISTERED, orderDAO.save, y logStatus(order, from, to, actor, "Tara registrada: <peso> kg"). Cualquier otra falla -> BusinessException con log.error.
- Devuelve la Order guardada.

NO toques: Order.java, ConflictException, Constants, ActivationPasswordGenerator (Etapa A); nada de integration (Etapa C); Postman ni docs/ (Etapa D). Si hace falta un cambio ajeno, avisame.

Reglas: metodología de la cátedra (capas, I*Business, excepciones checked); identificadores en inglés y comentarios en español. Commits atómicos de 100 líneas o menos, en español, formato tipo(ámbito): descripción, SIN Co-Authored-By de Claude ni "Generated with Claude Code". No hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase. No amplíes el alcance ni avances a otra etapa. Antes de editar, mostrame tu plan y esperá mi OK.

Verificá de verdad: compilá, y cuando Benjamín haya publicado el endpoint probá por HTTP con la base levantada: tara válida (200, la orden pasa a estado 2, historial 1->2), segunda tara (409), peso 0 y negativo (400), orden inexistente (404) y una falla simulada que deje la orden en estado 1 sin tara. Reportá solo lo que ejecutaste; si algo falla, decilo.
```

Etapa C — Endpoint de la balanza (TMS)

Responsable: Benjamín Alasino · **Estimación:** ≈2 h

Punto de partida (qué heredás de la Fase 1)

Existe `integration/sap/controllers/OrderSapRestController.java`, el modelo a imitar: extiende `BaseRestController`, usa `IOrderBusiness` y `IStandartResponseBusiness`, y traduce cada excepción a un código HTTP. **No existe el paquete `integration/tms`**: lo creás vos. Es el primer endpoint que usa la balanza.

Objetivo

Exponer `POST /api/v1/integration/tms/orders/{orderNumber}/tare`, que recibe la tara, llama a la regla de B y devuelve la contraseña.

Qué hacer, paso a paso

Esperá el push de la Etapa 1 de B (la firma de `registerTare`). Hasta entonces podés leer el controller de SAP y preparar las clases del pedido y la respuesta.

Paso 1 — Clases del pedido y la respuesta (paquete `integration/tms/model/`):

```
// TareRequest.java
@Getter @Setter @NoArgsConstructor
public class TareRequest {
    @Schema(description = "Peso del camión vacío, en kg", example = "14500.0")
    private double weight;
}

// TareResponse.java
@Getter @AllArgsConstructor
public class TareResponse {
    @Schema(description = "Contraseña de activación de 5 dígitos", example = "48213")
    private Integer activationPassword;
}
```

Paso 2 — El controller (`integration/tms/controllers/OrderTmsRestController.java`):

```
@RestController
@RequestMapping(Constants.URL_INTEGRATION_TMS + "/orders")
@Tag(name = "Integración TMS (balanza)", description = "Pesaje inicial y final de la orden")
public class OrderTmsRestController extends BaseRestController {

    @Autowired private IOrderBusiness orderBusiness;
    @Autowired private IStandartResponseBusiness response;

    @Operation(summary = "Registra la tara (pesaje inicial)", description = "...")
    @ApiResponse(responseCode = "200", description = "Tara registrada; devuelve la contraseña de activación")
    @ApiResponse(responseCode = "400", description = "Tara inválida")
    @ApiResponse(responseCode = "404", description = "La orden no existe")
    @ApiResponse(responseCode = "409", description = "La orden no está pendiente de pesaje inicial")
    @PostMapping(value = "/{orderNumber}/tare", consumes = MediaType.APPLICATION_JSON_VALUE,
            produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<?> registerTare(@PathVariable Integer orderNumber, @RequestBody TareRequest body) {
        try {
            User user = getUserLogged();
            Order o = orderBusiness.registerTare(orderNumber, body.getWeight(), user == null ? null :
user.getUsername());
            return new ResponseEntity<>(new TareResponse(o.getActivationPassword()), HttpStatus.OK);
        } catch (NotFoundException e) {   // 404
        } catch (BadRequestException e) { // 400
        } catch (ConflictException e) {   // 409
        } catch (BusinessException e) {   // 500
        }
    }
}
```

Completá cada `catch` igual que en `OrderSapRestController`: `new ResponseEntity<>(response.build(HttpStatus.X, e, e.getMessage()), HttpStatus.X)`.

Cómo probar

Con la app andando y la regla de B ya publicada:

```
TOKEN=$(curl -s -X POST localhost:8080/api/v1/login -d "username=admin&password=admin123")
# 1) crear una orden (Postman "Crear orden" o Swagger), por ejemplo la 90001
# 2) registrar la tara
curl -i -X POST localhost:8080/api/v1/integration/tms/orders/90001/tare \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{"weight":14500}'
```

Pedido	Esperado
Tara válida	200 y `{"activationPassword": <5 dígitos>}`
Mismo pedido otra vez	409
`{"weight":0}` o `{"weight":-5}`	400
`{}` (sin weight)	400
JSON malformado	400
Orden inexistente	404
Sin token	401
`orderNumber` no numérico (`/orders/abc/tare`)	400

Además, `http://localhost:8080/swagger-ui.html` tiene que mostrar el endpoint nuevo con sus respuestas.

Criterio de terminado

- La tabla de arriba se cumple.
- Swagger muestra el endpoint documentado.
- Pusheado a la rama y B revisó tu código.

No tocar

La regla de negocio (B), `Order.java`, `Constants` y `ConflictException` (A), Postman y docs (D).

Prompt para Claude Code (Etapa C)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 2 (pesaje inicial / tara), rama fase-2-pesaje-inicial. Somos 4 personas en la misma rama y cada archivo tiene un solo dueño. El equipo ya autorizó la fase: la línea de CLAUDE.md que dice "Fase 2 NO iniciada" quedó desactualizada; no la edites.

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md y docs/fase-2-division-de-trabajo.md (secciones 2, 3, 4 y "Etapa C"), y estudiá integration/sap/controllers/OrderSapRestController.java, que es el modelo a imitar.

Soy Benjamín Alasino y hago la ETAPA C (endpoint de la balanza TMS). Antes de empezar hacé git fetch, git checkout fase-2-pesaje-inicial y git pull --rebase: necesito que Martín (Etapa A) haya publicado ConflictException y Constants.URL_INTEGRATION_TMS, y que Tomás (Etapa B) haya publicado la firma Order registerTare(Integer orderNumber, double weight, String actor) en IOrderBusiness. Si no están, avisame y esperá.

Implementá SOLO estos archivos nuevos:
1. integration/tms/model/TareRequest.java: clase con double weight (Lombok @Getter/@Setter/@NoArgsConstructor, @Schema con ejemplo 14500.0).
2. integration/tms/model/TareResponse.java: clase con Integer activationPassword (@Getter/@AllArgsConstructor, @Schema).
3. integration/tms/controllers/OrderTmsRestController.java: @RestController en Constants.URL_INTEGRATION_TMS + "/orders", extiende BaseRestController, con @Tag, @Operation y @ApiResponse (200, 400, 404, 409). Método POST /{orderNumber}/tare (consumes y produces JSON) que obtiene el usuario con getUserLogged(), llama a orderBusiness.registerTare(orderNumber, body.getWeight(), username) y responde 200 con new TareResponse(order.getActivationPassword()). Traducción de errores con IStandartResponseBusiness.build(status, e, e.getMessage()): NotFoundException -> 404, BadRequestException -> 400, ConflictException -> 409, BusinessException -> 500. Mismo estilo que OrderSapRestController.

NO toques: IOrderBusiness ni OrderBusiness (Etapa B); Order.java, ConflictException, Constants (Etapa A); Postman ni docs/ (Etapa D). Si hace falta un cambio ajeno, avisame.

Reglas: metodología de la cátedra (capas, excepciones checked, StandartResponse, Constants); identificadores en inglés y comentarios en español. Commits atómicos de 100 líneas o menos, en español, formato tipo(ámbito): descripción, SIN Co-Authored-By de Claude ni "Generated with Claude Code". No hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase. No amplíes el alcance ni avances a otra etapa. Antes de editar, mostrame tu plan y esperá mi OK.

Verificá de verdad: compilá, arrancá la app con la base levantada, creá una orden (Postman "Crear orden" o Swagger) y probá con curl o Swagger: tara válida (200 con activationPassword de 5 dígitos), repetir la tara (409), weight 0, negativo y body vacío (400), JSON malformado (400), orden inexistente (404), sin token (401) y orderNumber no numérico (400). Comprobá que el endpoint aparece en /swagger-ui.html. Reportá solo lo que ejecutaste; si algo falla, decilo.
```

Etapa D — Pruebas, Postman y documentación

Responsable: Joaquín Contreras · **Estimación:** media jornada, repartida en la fase

Punto de partida (qué heredás de la Fase 1)

Hay una colección de Postman con **dos carpetas** (`00 - Autenticación` y `01 - Fase 1: recepción de la orden (SAP)`), 10 pedidos y 19 verificaciones que pasan con newman. La documentación de cada fase sigue el formato `docs/fase-N-<tema>.md` (informe técnico + guía de estudio A–J con al menos 10 preguntas) más su PDF. `CLAUDE.md` y la guía de `src` se actualizan al cerrar cada fase.

Objetivo

Probar la Fase 2 de punta a punta, dejar la colección de Postman con sus casos y documentar la fase.

Qué hacer, paso a paso

Paso 1 — Diseñar los casos (mientras esperás a B y C). Una carpeta nueva `02 - Fase 2: pesaje inicial (TMS)` con estos pedidos:

Pedido	Esperado
Crear una orden nueva para la prueba	201 (guardar su número en una variable propia, por ejemplo `orderNumberTms`)
Tara válida	200; la contraseña tiene 5 dígitos (10000–99999); se guarda en la variable `activationPassword`
Consultar la orden	`status` es 2; `tare` y `initialWeighingDateTime` presentes; **no** hay `activationPassword`
Historial de estados	2 registros: `null→1` y `1→2`; `actor` = usuario
Repetir la tara	409
Tara 0	400
Tara negativa	400
Sin `weight`	400
JSON malformado	400
Orden inexistente	404
Sin token	401

Usá **variables propias** (no `orderNumber`, que usa la Fase 1) para no pisar los pedidos anteriores.

Ejemplo de verificación para la tara válida:

```
pm.test('Status 200', () => pm.response.to.have.status(200));
const p = pm.response.json().activationPassword;
pm.test('La contraseña tiene 5 dígitos', () => pm.expect(p).to.be.within(10000, 99999));
pm.collectionVariables.set('activationPassword', String(p));
```

Paso 2 — Armar la carpeta en Postman cuando el endpoint de C esté publicado. Exportá la colección (v2.1) a `docs/postman/IW3-Gas-Terminal.postman_collection.json` y revisá con `git diff` que **solo cambie lo nuevo**. Va en un commit aparte.

Paso 3 — Prueba de punta a punta (Etapa 3, con los 4).

```
npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json
```

Tiene que pasar **todo**, incluida la carpeta de la Fase 1 (sus 19 verificaciones no se pueden romper).

Paso 4 — Documentación.

1. Crear `docs/fase-2-pesaje-inicial.md` con el formato de la Fase 1: informe técnico (archivos, funciones, recorrido del pedido, pruebas ejecutadas con resultados) y guía de estudio A–J con **10 preguntas o más**.
2. Pedir a A, B y C un bloque de texto con **sus archivos** (archivo, acción, motivo, responsabilidad) y 2 o 3 preguntas del profesor con respuesta. Vos los integrás.
3. Actualizar `CLAUDE.md` (secciones 2, 5, 7, 8 y 10), `docs/plan-de-trabajo.md` (decisiones nuevas) y `docs/guia-estudio-src.md` (archivos nuevos, marcados como F2).
4. Regenerar los PDF:

```
node docs/pdf/md2pdf.js docs/fase-2-pesaje-inicial.md docs/pdf/fase-2-pesaje-inicial.pdf "Fase 2 · Pesaje inicial"
node docs/pdf/md2pdf.js docs/guia-estudio-src.md docs/pdf/guia-estudio-src.pdf "Guía de estudio del código"
```

Paso 5 — Abrir el Pull Request de la rama a `main` (como borrador, con la descripción de los cambios y cómo se probó). Se mergea con la aprobación de 3 de los 4 integrantes.

Criterio de terminado

- La carpeta de la Fase 2 pasa con newman y la de la Fase 1 sigue pasando.
- Informe y guía completos, con las partes de A, B y C integradas.
- `CLAUDE.md`, plan y guía de `src` actualizados; PDF regenerados.
- PR abierto.

No tocar

Código de `src/main` (A, B, C). Si en las pruebas encontrás un error, **avisale a su dueño** con el pedido exacto y la respuesta que obtuviste.

Prompt para Claude Code (Etapa D)

```
Contexto: repo iw3-gas-terminal-backend (Spring Boot 4, Java 21, MySQL). Estamos en la Fase 2 (pesaje inicial / tara), rama fase-2-pesaje-inicial. Somos 4 personas en la misma rama y cada archivo tiene un solo dueño. El equipo ya autorizó la fase: la línea de CLAUDE.md que dice "Fase 2 NO iniciada" quedó desactualizada y la corrijo yo al final de la fase.

Leé primero: CLAUDE.md, docs/plan-de-trabajo.md, docs/fase-1-recepcion-ordenes.md (como molde del informe y la guía) y docs/fase-2-division-de-trabajo.md (secciones 2, 3, 4 y "Etapa D"). Revisá también la colección docs/postman/IW3-Gas-Terminal.postman_collection.json para imitar su estilo.

Soy Joaquín Contreras y hago la ETAPA D (pruebas, Postman y documentación). Antes de empezar hacé git fetch, git checkout fase-2-pesaje-inicial y git pull --rebase.

Tu trabajo, en este orden:
1. Mientras B y C no hayan publicado, diseñá los casos de Postman. Cuando el endpoint POST /api/v1/integration/tms/orders/{orderNumber}/tare esté publicado, agregá a docs/postman/IW3-Gas-Terminal.postman_collection.json una carpeta "02 - Fase 2: pesaje inicial (TMS)" con: crear orden nueva para la prueba (guardar su número en una variable PROPIA, no orderNumber), tara válida (200, activationPassword de 5 dígitos entre 10000 y 99999, guardarlo en una variable), consultar la orden (status 2, tare e initialWeighingDateTime presentes y SIN activationPassword), historial (2 registros: null->1 y 1->2, con actor), repetir la tara (409), tara 0 (400), tara negativa (400), sin weight (400), JSON malformado (400), orden inexistente (404) y sin token (401). Cada pedido con sus tests. No modifiques los pedidos de la Fase 1. Commit del JSON aparte, solo ese archivo, y verificá con git diff que solo cambie lo nuevo.
2. Cuando las tres etapas estén integradas, corré todo con npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json contra la app y la base reales. Tiene que pasar la carpeta de la Fase 2 y las 19 verificaciones de la Fase 1.
3. Documentación: creá docs/fase-2-pesaje-inicial.md con el formato de docs/fase-1-recepcion-ordenes.md (informe técnico + guía de estudio A-J con 10 preguntas o más), integrando los bloques que me pasen A, B y C; actualizá CLAUDE.md (secciones 2, 5, 7, 8 y 10), docs/plan-de-trabajo.md (decisiones nuevas: 409, contraseña oculta, orden de comprobaciones) y docs/guia-estudio-src.md (archivos nuevos marcados F2). Regenerá los PDF con node docs/pdf/md2pdf.js (ver docs/pdf/README.md).

NO toques código de src/main ni src/test (Etapas A, B y C). Si una prueba falla, no lo arregles vos: decime el pedido exacto, la respuesta obtenida y a qué etapa corresponde, para avisarle a su dueño. No commitees la carpeta .postman/.

Reglas: documentación y comentarios en español, identificadores en inglés. Commits atómicos de 100 líneas o menos (el JSON de Postman es la excepción: un commit aparte), en español, formato tipo(ámbito): descripción, SIN Co-Authored-By de Claude ni "Generated with Claude Code". No hagas push, merge ni force push salvo que yo lo pida; cuando lo pida, primero git pull --rebase. No amplíes el alcance. Reportá solo lo que ejecutaste; si algo falla, decilo. Antes de editar, mostrame tu plan y esperá mi OK.
```

Cierre de la fase

Checklist de integración (Etapa 4)

✓	Verificación	Quién
☐	`./mvnw -B -DskipTests compile` sin errores en la rama	Todos
☐	`./mvnw -B test` pasa (necesita la base)	A
☐	La app arranca contra una base que ya tenía órdenes	A
☐	Los 11 criterios de aceptación de la sección 2	D
☐	newman: carpeta de la Fase 2 y Fase 1 en verde	D
☐	Revisión cruzada hecha (A→B, B→C, C→D, D→A)	Todos
☐	Cada uno puede explicar el recorrido completo de la tara	Todos
☐	PR abierto como borrador, sin `.postman/`	D
☐	3 aprobaciones antes de mergear	Equipo
☐	Después del merge: reconstruir la imagen y redesplegar en la VM	Equipo

Problemas frecuentes

Síntoma	Causa probable	Qué hacer
El push es rechazado	Alguien pusheó antes	`git pull --rebase` y volver a pushear
No compila: no encuentra `registerTare` o `ConflictException`	Falta traer el push de A o de B	`git pull --rebase`
Conflicto de Git en un archivo	Dos personas editaron el mismo archivo	No resolver a ciegas: avisar a los dos dueños
500 en vez de 409 o 400	La comprobación quedó dentro del `try/catch`	Moverla fuera del `try` (Etapa B)
Error de columna al arrancar	El campo nuevo no es nulable	Usar `Double` e `Integer` (Etapa A)
`GET /orders` muestra la contraseña	Falta `@JsonIgnore`	Etapa A
Falla una verificación de la Fase 1 en newman	Se cambió algo que usa la Fase 1	Avisar al dueño del cambio
401 en todo	Token vencido (dura 1 hora) o falta el usuario de prueba	Volver a loguearse / correr el seed
El contenedor de la base no arranca	Docker apagado o nombre en conflicto	Iniciar Docker; el contenedor del TP es `iw3-gas-mysql`

Después de la Fase 2

Siguen la Fase 3 (activación con la contraseña y datos del caudalímetro en tiempo real), la 4 (cierre) y la 5 (pesaje final y conciliación). La contraseña que se genera acá es la que valida la Fase 3.